TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

JONATHAN ENRIQUE YUGCHA

PARALELO:

"A"

SEMESTRE:

QUINTO

ACTIVIDAD:

Actividad 1

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad _1_

1. Introducción

El rescate y la adopción de animales requieren una organización eficiente para registrar los casos reportados, controlar el estado de los animales rescatados, asignar responsables y facilitar los procesos de atención veterinaria y adopción.

En muchos refugios, estos procedimientos pueden gestionarse mediante registros manuales, hojas de cálculo o comunicaciones informales, lo que dificulta el seguimiento de cada caso y la coordinación entre ciudadanos, voluntarios, veterinarios y administradores.

Por esta razón, se propone desarrollar un Sistema de Control de Rescate y Adopción Animal, una solución informática que permita centralizar la información de los animales, registrar sus necesidades, asignar responsables y mantener un historial de las actividades realizadas.

El diseño se fundamentará en principios de programación orientada a objetos, principios SOLID y patrones de diseño de software. Además, se utilizará GitHub con la metodología Gitflow para organizar el desarrollo, controlar las versiones y facilitar la incorporación de nuevas funcionalidades

2. Objetivos

Objetivo general

Diseñar la arquitectura lógica de un sistema de gestión de rescate y adopción animal mediante la aplicación de patrones de diseño de software, diagramas UML y el flujo de trabajo Gitflow, con la finalidad de construir una solución escalable, reutilizable y mantenible.

Objetivos específicos

- Identificar los actores involucrados y los requerimientos funcionales y no funcionales del sistema.
- Diseñar diagramas UML que representen las interacciones, clases y secuencias principales de la aplicación.

- Aplicar patrones de diseño como MVC, Factory Method, Observer y Singleton para organizar las responsabilidades del sistema.
- Establecer una arquitectura lógica basada en la separación de responsabilidades y los principios SOLID.
- Configurar un repositorio público en GitHub utilizando Gitflow para gestionar ramas, funcionalidades y versiones.
- Documentar las decisiones técnicas y justificar los patrones de diseño seleccionados.

Actores principales

El sistema tendrá cuatro actores principales.

Actor	Descripción	Responsabilidades
Ciudadano o reportante	Persona que identifica un animal que necesita ayuda.	Registrar un reporte de rescate y consultar el estado de su reporte.
Voluntario	Persona encargada de colaborar en las actividades de rescate y cuidado.	Consultar casos asignados, actualizar avances y registrar actividades.
Veterinario	Profesional responsable de la atención médica.	Evaluar animales, registrar tratamientos y actualizar su estado clínico.
Administrador del refugio	Persona encargada de supervisar las operaciones.	Gestionar casos, asignar responsables, administrar expedientes y supervisar adopciones.

Requerimientos funcionales:

Los requerimientos funcionales describen las operaciones que el sistema debe permitir.

- **Registrar reportes:** Permitir que un ciudadano registre un caso de rescate.
- **Categorizar especies:** Clasificar los animales como perros, gatos o fauna silvestre
- **Crear expedientes:** Generar un expediente con los datos de cada animal rescatado

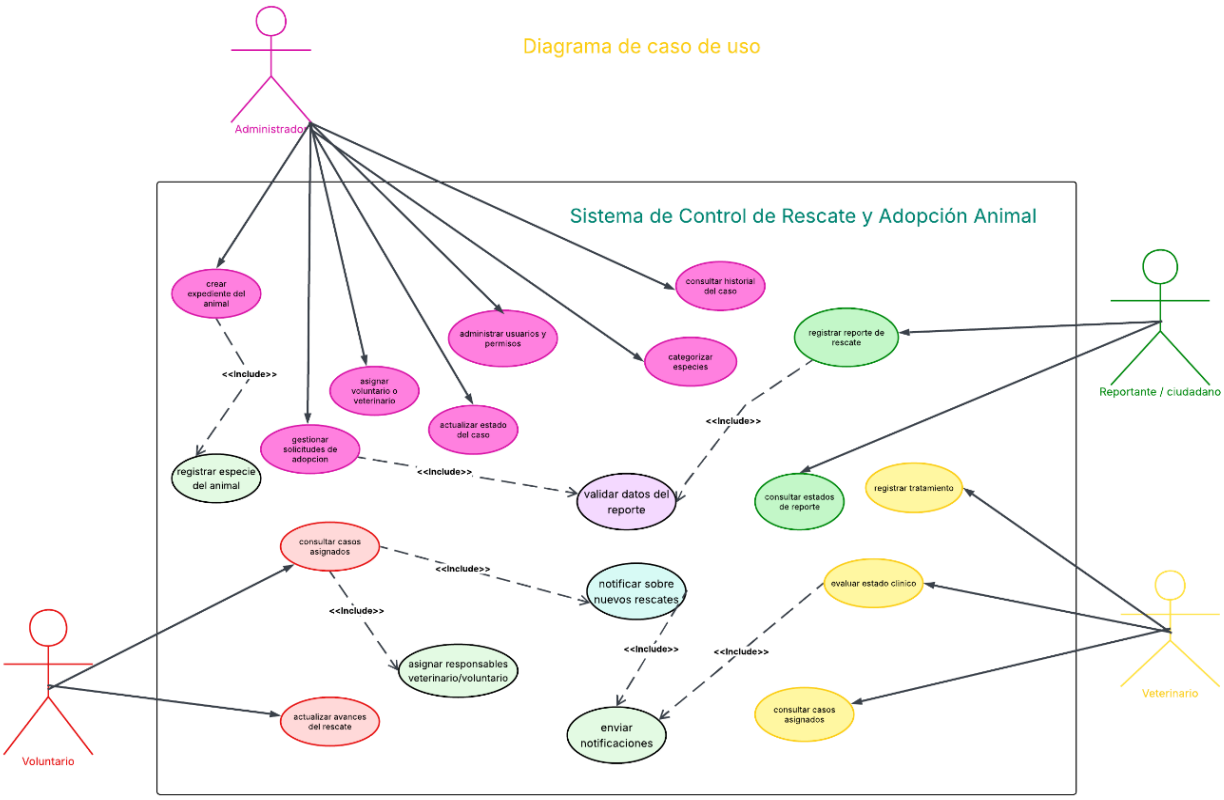
- **Asignar responsables:** Permitir al administrador asignar voluntarios o veterinarios.
- **Gestionar estados:** Actualizar el estado del caso según su evolución
- **Registrar tratamientos:** Guardar evaluaciones veterinarias y tratamientos realizados.
- **Notificar responsables:** Enviar notificaciones cuando se registren casos nuevos o se produzcan cambios importantes
- **Gestionar adopciones:** Registrar solicitudes, evaluar candidatos y formalizar adopciones
- **Consultar expedientes:** Buscar animales por identificador, especie o estado.
- **Consultar historial:** Mantener un registro de las modificaciones y actividades realizadas.
- **Administrar usuarios:** Gestionar las cuentas, roles y permisos del personal
- **Controlar disponibilidad:** Identificar animales disponibles para adopción

Requerimientos No Funcionales

- **Escalabilidad:** Permitir incorporar nuevos tipos de animales y funcionalidades
- **Mantenibilidad:** Organizar el código en módulos con responsabilidades claramente definidas.
- **Seguridad:** Controlar el acceso mediante autenticación y autorización por roles.
- **Trazabilidad:** Registrar los cambios del sistema y mantener un historial de versiones.
- **Usabilidad:** Ofrecer una interfaz clara y fácil de utilizar.
- **Fiabilidad:** Validar la información para evitar registros inconsistentes
- **Reutilización:** Utilizar componentes y patrones que puedan aprovecharse en diferentes módulos.
- **Integridad de datos:** Evitar que los cambios de estado produzcan información contradictoria.

Diagrama de caso de uso

Crear un diagrama de casos de uso para entender las interacciones del personal con la plataforma.



Este diagrama muestra las operaciones disponibles para los diferentes actores, el ciudadano puede registrar reportes y consultar su estado. El voluntario y el veterinario trabajan con los casos que tienen asignados, mientras que el administrador supervisa los expedientes, las asignaciones y las adopciones.